

Name: J.Mohamed Aseem

Reg No:192511258

Date:18/07/2026

Hackerone

1. Organization Name

HackerOne (Self-compiled Platform Bug)

2. Vulnerability Title

New Hacktivity features: Bounty rewards leakage where programs choose not to disclose bounty amounts in limited disclosure reports.

3. Vulnerability Category

Information Disclosure / Business Logic Flaw (Broken Access Control)

4. Brief Description

The security researcher identified a logical flaw introduced by a newly rolled-out search feature on the HackerOne Hacktivity dashboard. While some organizations choose a "limited disclosure" policy—meaning the public can see the vulnerability title but the financial bounty amount awarded to the hacker is explicitly hidden—the new dashboard implemented an interactive search filter called `total_awarded_amount`. By applying custom parameters (e.g., searching specifically for `total_awarded_amount:10000`), an attacker or public user could systematically guess and infer the exact hidden bounty amounts of restricted reports based on whether they appeared in the filtered results.

5. What Was the Security Impact?

- **Loss of Confidentiality:** It completely bypassed the privacy controls set by both companies and independent hackers who intentionally opted to keep their financial earnings private.
- **Trust Disruption:** Organizations rely on platform confidentiality guarantees to run private or semi-private bug bounty operations. Leaking financial structures can expose an organization's internal security budget allocations to competitors or malicious actors.

6. Disclosure Process Followed

The security researcher discovered the operational loophole and submitted a detailed report directly to HackerOne's own security team via their platform. The team triaged the issue, verified that the backend elastic-search filters were exposing data that front-end privacy rules

meant to hide, implemented a patch to restrict queries on limited reports, and mutually agreed on public disclosure once fixed.

7. Was Responsible Disclosure Followed?

Yes. The researcher adhered strictly to HackerOne's vulnerability disclosure guidelines. The vulnerability details, replication steps, and proof-of-concept filters were kept confidential between the researcher and the security team until a full security patch was successfully deployed.

8. Could Full Disclosure Have Been Dangerous?

Yes, moderately dangerous. Had the researcher published this actively before it was patched (Zero-Day disclosure), malicious parties could have mass-scraped the entire historical database of HackerOne's limited disclosures. This would have created a permanent public archive mapping corporate security vulnerabilities to specific cash values, giving adversaries immediate intelligence on which corporate assets are historically weak and high-value targets.

9. Lessons Learned

- **Feature Creep Implies Risk Creep:** When building new user-experience metrics or search functionality (like advanced filtering tools), backend APIs must re-validate existing privacy settings.
- **Data Aggregation Queries:** Simply hiding an attribute on the user interface (UI) is insufficient if the query mechanism allows filtering by that same hidden attribute. True data isolation must happen at the database query layer.

10. Conclusion

This case study highlights the critical nature of access control verification during the release of new application features. By identifying a business logic flaw in the search filter mechanics, the researcher protected the privacy choices of hundreds of enterprises and security professionals. The swift patch by the organization reinforces why structured bug bounty frameworks are vital to maintaining modern software infrastructure safety.

Name: J. Mohamed Aseem

Reg No: 192511258

Date: 18/07/2026

CVE Selected: CVE-2025-55182 (React2Shell)

CVE

1. Introduction to CVE

The Common Vulnerabilities and Exposures (CVE) system is a catalog of publicly disclosed cybersecurity vulnerabilities managed by the MITRE Corporation. Each entry provides a unique identifier, a description, and at least one public reference. The National Vulnerability Database (NVD), operated by the National Institute of Standards and Technology (NIST), synchronizes with the CVE list to perform deep analysis. The NVD assigns severity scores using the Common Vulnerability Scoring System (CVSS), determines the underlying structural flaw via Common Weakness Enumeration (CWE), and maps affected software stacks.

2. Vulnerability Description

- **Vulnerability Name:** React2Shell
- **CVE ID:** CVE-2025-55182
- **Affected Software:** Meta React (Specifically React 19 frameworks employing React Server Components, such as Next.js)
- **CVSS v3.1 Severity Score:** 10.0 (Critical)
- **Vector String:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
- **Associated CWE:** CWE-94 (Improper Control of Generation of Code / Code Injection)

Technical Summary:

The vulnerability exists within the application layer handling React Server Components (RSCs). When an application is deployed in specific server-side rendering (SSR) configurations, the framework processes serialized payloads sent by the client. Due to insufficient input sanitization and insecure handling of component property bindings, a remote, unauthenticated attacker can pass malicious, encoded inputs through an HTTP request. The backend server parses these inputs as trusted executable server instructions rather than isolated data properties. This results in arbitrary code execution directly on the underlying node application environment host.

3. Disclosure Timeline

The coordination window for this vulnerability progressed as follows:

- **October 14, 2025:** Security researchers discover the flaw during a routine audit of production Next.js application deployments and secretly notify Meta's security team with a working Proof of Concept (PoC).
- **November 2, 2025:** Meta security engineering teams confirm the logic flaw in the internal serialization parsing module and begin drafting core patches.
- **December 3, 2025:** The patch is finalized, integrated into upstream builds, and a coordinated public advisory is issued alongside the official registration of CVE-2025-55182.
- **December 5, 2025:** The Cybersecurity and Infrastructure Security Agency (CISA) adds the vulnerability to its Known Exploited Vulnerabilities (KEV) catalog due to observed active exploitation attempts in the wild.

4. Analysis

The architectural design of React Server Components (RSC) aims to merge frontend interface components with backend data queries seamlessly. However, this design inadvertently expands the attack surface.

The underlying issue stems from a structural failure to isolate client data from server execution context. Because React is the dominant frontend architecture globally, the attack surface is vast. Statistics collected immediately after disclosure indicated that scanning tools encountered more exposed enterprise applications for React2Shell than for the historic Log4Shell vulnerability, making it an incredibly high-impact vector for enterprise networks.

5. Responsible Disclosure Evaluation

The reporting party closely adhered to the industry standard **Responsible Disclosure** doctrine. Instead of pursuing full public disclosure immediately (which would have exposed millions of websites to zero-day mass exploitation), the researchers submitted the data through private channels.

- **Could Full Disclosure Have Been Dangerous?** Yes, catastrophically dangerous. Because the exploit requires low technical complexity, no authentication credentials, and zero user interaction, a premature public disclosure would have enabled automated malware frameworks to launch automated remote code execution scripts, leading to massive data breaches and systemic host takeovers worldwide before patches could be deployed.

6. Conclusion

CVE-2025-55182 represents a significant paradigm shift in web application vulnerabilities. It highlights how modern full-stack web architectures, which abstract the boundary between frontend clients and server-side execution, can introduce severe logic flaws if deserialization processes are not strictly sandboxed. By leveraging the structured definitions provided by CVE and the analytical metrics of the NVD registry, engineering teams were able to identify, track, and roll out mandatory hotfixes to secure web infrastructure globally.

